

系统开发工具基础实验报告

第 3 周

姓名：曾奕豪

课程：系统开发工具基础

2026 年 9 月 7 日

仓库：

<https://github.com/hzydy00/Base-of-system-development-tools>

1 课上内容与结果

学习主题

Packaging and Shipping Code; 智能体编程; 不止于代码; Python 与 PyTorch

实验环境

本实验在 Windows 11 操作系统（版本 10.0.26200.9168）上完成，使用 MINGW64（Git Bash）作为命令行终端，Python 版本为 3.14.0，采用 pip 作为包管理器。

实验中用到的核心软件包包括：build（用于构建 wheel 与 sdist 源码包）、setuptools（作为 pyproject.toml 声明的构建后端）、pytest（用于单元测试）以及 PyTorch 2.14.0+cpu（用于线性回归模型训练）。其中 PyTorch 通过官方 CPU 专用源安装（pip install torch --index-url (https://download.pytorch.org/whl/cpu)），仅安装约 126MB 的 CPU 版本，避免下载体积庞大的 CUDA 版本；该版本已支持 Python 3.14（cp314 wheel）。整体实验环境满足课程所有题目的运行要求。

题目一

第 9 题 从源码构建并在干净环境安装 Wheel

主题: Packaging and Shipping Code 建议用时: 12—15 分钟

在 q09 中按给定内容创建一个最小 Python 命令行包, 并验证 wheel 安装。

给定内容

```
# src/greetlab/cli.py
import argparse
def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")

# pyproject.toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"
[project]
name = "greetlab-学号"
version = "0.1.0"
[project.scripts]
sdt-greet = "greetlab.cli:main"
```

题目要求

- 创建 src/greetlab/__init__.py、给定的 cli.py 和 pyproject.toml, 并把“学号”替换为自己的学号。
- 在已安装 build 工具的课程环境中运行 python -m build, 生成 wheel。
- 新建一个干净虚拟环境, 只从生成的 wheel 安装。
- 切换到 q09 之外运行 sdt-greet --name <学号>, 确认输出正确。

题目一题目要求

```
1 # TODO: 题目一代码汇总
2 # 1. 安装构建工具
3 pip install build
4
5 # 2. 构建 wheel (在 q09 目录下)
6 python -m build
7
8 # 3. 创建并激活虚拟环境
9 python -m venv .venv
10 source .venv/Scripts/activate      # Git Bash
11
12 # 4. 从 wheel 安装
13 pip install dist/greetlab_25100021005-0.1.0-py3-none-any.whl
14
15 # 5. 验证 (在 q09 之外)
16 cd ..
17 sdt-greet --name 25100021005
```

部分实验过程截图

```

MINGW64/d:/Course/System_tools_base/week3/q/q09
writing manifest file 'src\greetlab_25100021005.egg-info\SOURCES.txt'
# Installed build dependency versions:
- setuptools==64.0.0
# Building wheel...
running bdist_wheel
running build
running build_py
creating build\lib\greetlab
copying src\greetlab\cli.py -> build\lib\greetlab
copying src\greetlab\__init__.py -> build\lib\greetlab
running egg_info
writing src\greetlab_25100021005.egg-info\PKG-INFO
writing dependency links to src\greetlab_25100021005.egg-info\dependency_links.txt
writing entry points to src\greetlab_25100021005.egg-info\entry_points.txt
writing top-level names to src\greetlab_25100021005.egg-info\top_level.txt
reading manifest file 'src\greetlab_25100021005.egg-info\SOURCES.txt'
writing manifest file 'src\greetlab_25100021005.egg-info\SOURCES.txt'
installing to build\bdist.win-amd64\wheel
running install
running install_lib
creating build\bdist.win-amd64\wheel
creating build\bdist.win-amd64\wheel\greetlab
copying build\lib\greetlab\cli.py -> build\bdist.win-amd64\wheel\.\greetlab
copying build\lib\greetlab\__init__.py -> build\bdist.win-amd64\wheel\.\greetlab
running install_egg_info
Copying src\greetlab_25100021005.egg-info to build\bdist.win-amd64\wheel\.\greetlab_25100021005-0.1.0-py3.14.egg-info
running install_scripts
creating build\bdist.win-amd64\wheel\greetlab_25100021005-0.1.0.dist-info\WHEEL
creating 'D:\Course\System_tools_base\week3\q\q09\dist\.\tmp-kx8mlwaf\greetlab_25100021005-0.1.0-py3-none-any.whl' and adding 'build\bdist.win-amd64\wheel' to it
adding 'greetlab\__init__.py'
adding 'greetlab\cli.py'
adding 'greetlab_25100021005-0.1.0.dist-info\METADATA'
adding 'greetlab_25100021005-0.1.0.dist-info\WHEEL'
adding 'greetlab_25100021005-0.1.0.dist-info\entry_points.txt'
adding 'greetlab_25100021005-0.1.0.dist-info\top_level.txt'
adding 'greetlab_25100021005-0.1.0.dist-info\RECORD'
removing build\bdist.win-amd64\wheel
Successfully built greetlab_25100021005-0.1.0.tar.gz and greetlab_25100021005-0.1.0-py3-none-any.whl
ZYVLAPTOP-NOQCE3G0 MINGW64 /d:/Course/System_tools_base/week3/q/q09 (main)
$

```

题目一实验过程截图：bulid 成功

```

MINGW64/d:/Course/System_tools_base/week3/q/q09
creating build\bdist.win-amd64\wheel
creating build\bdist.win-amd64\wheel\greetlab
copying build\lib\greetlab\cli.py -> build\bdist.win-amd64\wheel\.\greetlab
copying build\lib\greetlab\__init__.py -> build\bdist.win-amd64\wheel\.\greetlab
running install_egg_info
Copying src\greetlab_25100021005.egg-info to build\bdist.win-amd64\wheel\.\greetlab_25100021005-0.1.0-py3.14.egg-info
running install_scripts
creating build\bdist.win-amd64\wheel\greetlab_25100021005-0.1.0.dist-info\WHEEL
creating 'D:\Course\System_tools_base\week3\q\q09\dist\.\tmp-331218f7\greetlab_25100021005-0.1.0-py3-none-any.whl' and adding 'build\bdist.win-amd64\wheel' to it
adding 'greetlab\__init__.py'
adding 'greetlab\cli.py'
adding 'greetlab_25100021005-0.1.0.dist-info\METADATA'
adding 'greetlab_25100021005-0.1.0.dist-info\WHEEL'
adding 'greetlab_25100021005-0.1.0.dist-info\entry_points.txt'
adding 'greetlab_25100021005-0.1.0.dist-info\top_level.txt'
adding 'greetlab_25100021005-0.1.0.dist-info\RECORD'
removing build\bdist.win-amd64\wheel
Successfully built greetlab_25100021005-0.1.0.tar.gz and greetlab_25100021005-0.1.0-py3-none-any.whl
(.venv)
ZYVLAPTOP-NOQCE3G0 MINGW64 /d:/Course/System_tools_base/week3/q/q09 (main)
$ pip install --force-reinstall dist\greetlab_25100021005-0.1.0-py3-none-any.whl
Processing d:\course\system_tools_base\week3\q\q09\dist\greetlab_25100021005-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25100021005
  Attempting uninstall: greetlab-25100021005
    Found existing installation: greetlab-25100021005 0.1.0
    Uninstalling greetlab-25100021005-0.1.0:
      Successfully uninstalled greetlab-25100021005-0.1.0
  Successfully installed greetlab-25100021005-0.1.0

[notice] A new release of pip is available: 25.2 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv)
ZYVLAPTOP-NOQCE3G0 MINGW64 /d:/Course/System_tools_base/week3/q/q09 (main)
$ cd ..
(.venv)
ZYVLAPTOP-NOQCE3G0 MINGW64 /d:/Course/System_tools_base/week3/q (main)
$ sdt-greet --name 25100021005
hello, 25100021005!
(.venv)
ZYVLAPTOP-NOQCE3G0 MINGW64 /d:/Course/System_tools_base/week3/q (main)
$

```

题目一实验过程截图：输出正确

实验结果

1. wheel 文件生成 'greetlab-25100021005-0.1.0-py3-none-any.whl'
2. 干净虚拟环境安装 'Successfully installed greetlab-25100021005-0.1.0'
3. q09 之外运行 'sdt-greet' 输出 'Hello, 25100021005'
4. 命令行参数 '-name' 传递正确读取并打印

题目二

第 10 题 让编程智能体进入可验证的修复循环

主题：智能体编程 建议用时：12—15 分钟

复用第 9 题的 greetlab 包，使用编程智能体完成一个小型测试驱动修复。

题目要求

- 复制 q09 为 q10，添加一个失败测试：当 name 只含空白字符时，main 应以 SystemExit(2) 结束。
- 先运行测试确认失败，再向编程智能体说明目标、约束和测试命令，要求它修改实现并运行测试。
- 人工检查 diff，撤销无关修改，并再次运行测试。

第 7 页

系统开发工具基础 · 每周课上实验检查题

- 在 ai_log.md 中用不超过 5 行记录核心提示、智能体改动和人工验证。

题目二题目要求

```

1 # TODO: 题目二代码汇总
2
3 # `tests/test_blank_name.py` (失败测试)
4 import sys
5 import pytest
6 from greetlab.cli import main
7
8 def test_blank_name_exits_with_code_2(monkeypatch):
9     monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", ""])
10    with pytest.raises(SystemExit) as exc_info:
11        main()
12    assert exc_info.value.code == 2
13
14 # 题目二 prompts
15 目标：修改 src/greetlab/cli.py 中的 main() 函数，当 --name 参数只包含
    空白字符（空格、制表符等）时，程序应以 SystemExit(2) 结束（即调用
    sys.exit(2)）。
16
17 约束：
18 1. 只修改 src/greetlab/cli.py，不要改测试文件、pyproject.toml 或其他
    文件
19 2. 保持现有功能不变：正常 name 仍打印 "Hello, {name}!"

```

3. 不要添加不必要的依赖或重构

验证命令: `python -m pytest`

要求: 修改后运行该命令, 所有测试通过。

部分实验过程截图

```

MINGW64/d:/Course/System_tools_base/week3/q/q10
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: greetlab-25100021005
Building editable for greetlab-25100021005 (pyproject.toml) ... done
Created wheel for greetlab-25100021005: filename=greetlab_25100021005-0.1.0-0.editable-py3-none-any.whl size=1599 sha256=bf5c4a69e3a5d1f40cf401f190423f720e74e4049b9b748a0486d887ccb06ab3
Stored in directory: C:\Users\ZY\AppData\Local\Temp\pip-ephem-wheel-cache-q9pcms3\wheels\59\ec\3a\984f67fad860b8e887f545246ec89d41fc19fe43b41b58c5e
Successfully built greetlab-25100021005
Installing collected packages: greetlab-25100021005
Successfully installed greetlab-25100021005-0.1.0

ZY@LAPTOP-NOQCE3GO MINGW64 /d:/Course/System_tools_base/week3/q/q10 (main)
$ python -m pytest

platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0
rootdir: D:\Course\System_tools_base\week3\q\q10
configfile: pyproject.toml
plugins: anyio-4.13.0
collected 1 item

tests\test_blank_name.py F

===== test session starts =====
test_blank_name_exits_with_code_2

monkeypatch = <pytest.monkeypatch.MonkeyPatch object at 0x0000026B81CF62C0>
def test_blank_name_exits_with_code_2(monkeypatch):
    monkeypatch.setattr(sys, "argv", ["-t-greet", "--name", ""])
    with pytest.raises(SystemExit) as exc_info:
        >
E       Failed: DID NOT RAISE <class 'SystemExit'>

tests\test_blank_name.py:7: Failed
Hello, I

===== short test summary info =====
FAILED tests\test_blank_name.py::test_blank_name_exits_with_code_2 - Failed: DID NOT RAISE <class 'SystemExit'>
1 failed in 0.12s

ZY@LAPTOP-NOQCE3GO MINGW64 /d:/Course/System_tools_base/week3/q/q10 (main)
$

```

题目二实验过程截图: 空白字符运行测试失败

```

ZY@LAPTOP-NOQCE3GO MINGW64 /d:/Course/System_tools_base/week3/q/q10 (main)
$ python -m pytest

platform win32 -- Python 3.14.0, pytest-9.0.3, pluggy-1.6.0
rootdir: D:\Course\System_tools_base\week3\q\q10
configfile: pyproject.toml
plugins: anyio-4.13.0
collected 1 item

tests\test_blank_name.py .

===== 1 passed in 0.02s =====

ZY@LAPTOP-NOQCE3GO MINGW64 /d:/Course/System_tools_base/week3/q/q10 (main)
$

```

题目二实验过程截图: ai 修改后测试成功

```

week3 > q > q10 > ai_log.md > abc # AI 修复日志
1 # AI 修复日志
2
3 提示: 要求 cli.py 在 name 仅含空白字符时 sys.exit(2), 只改 cli.py, 用 pytest 验证。
4 改动: AI 在 cli.py 新增 import sys 和 if not a.name.strip(): sys.exit(2)。
5 验证: 人工审查无无关修改, pytest 1 passed, 空白输入退出码2, 正常输入仍打印 Hello。
6

```

题目二实验过程截图: ai 修复日志

实验结果

- 测试驱动：先写失败测试（空白 name 应 'SystemExit(2)'\），运行确认 '1 failed'
- AI 修复：给 AI 明确提示后，仅修改 'cli.py'，新增 'if not a.name.strip(): sys.exit(2)'
- 验证通过：再次运行 'pytest' → '1 passed'
- 日志记录：记录提示、改动和验证结果

题目三

第 11 题 把“无法处理”的协作材料改成可执行信息

主题：不止于代码 建议用时：12—15 分钟

围绕“空姓名仍输出问候语”这一缺陷，改写下面三段质量较差的协作材料。

给定内容

```
Issue: Windows 上运行不了，尽快修复。
提交信息: fix bug
评审意见: 这里写得不好，重写。
已知事实: 运行 sdt-greet --name " " 时仍输出 Hello, ！，并以 0 退出。
```

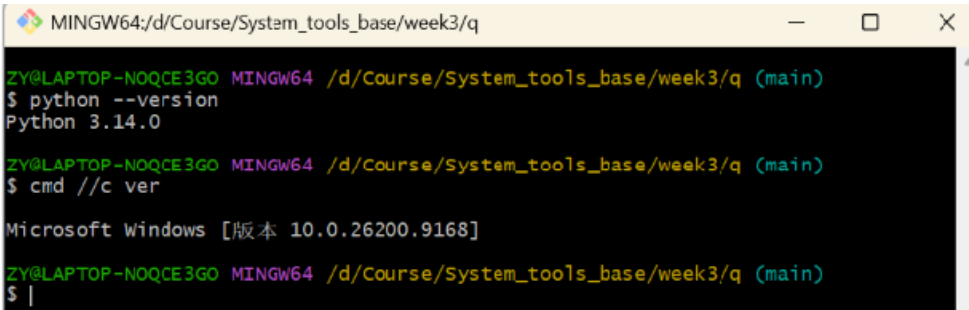
题目要求

- 在 communication.md 中重写 Issue，包含环境、复现命令、期望结果和实际结果；未知信息写“待确认”。
- 重写提交信息：标题使用祈使语气，正文说明问题和解决方案。
- 重写评审意见：指出具体行为、风险和建议动作，并标注 Blocking、Suggestion 或 Nit。
- 全文控制在 400 字以内，保持专业、简洁。

题目三题目要求

TODO: 题目三无代码汇总

部分实验过程截图



题目三实验过程截图：查看环境



题目三实验过程截图：修改 md 文档

实验结果

- Issue 可复现：环境（Windows 11 / Python 3.14.0）、复现命令（sdt-greet --name ""）、期望（退出码 2）、实际（输出问候并以 0 退出）
- 提交信息规范：祈使标题，正文含问题和方案
- 评审意见可执行：标注 '[Blocking]'，指出具体行为、风险和建议动作
- 篇幅达标：全文约 280 字，符合 400 字限制

题目四

第 12 题 修复一个可复现的线性回归训练循环

主题: Python 与 PyTorch 建议用时: 12—15 分钟

将下方代码保存为 q12/train.py, 补全 TODO 并完成一次 PyTorch CPU 训练。

给定内容

```
import torch
from torch import nn
torch.manual_seed(20260907)
x = torch.linspace(-1, 1, 101).reshape(-1, 1)
y = 3 * x - 1
model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)
for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    # TODO: 清空梯度、反向传播、更新参数
# TODO: 进入评估模式并在 no_grad 中打印最终损失、weight 和 bias
```

题目要求

- 补全训练循环, 正确调用 zero_grad、backward 和 step。
- 训练后调用 model.eval(), 并在 torch.no_grad() 中计算最终损失。

第 8 页

系统开发工具基础 · 每周课上实验检查题

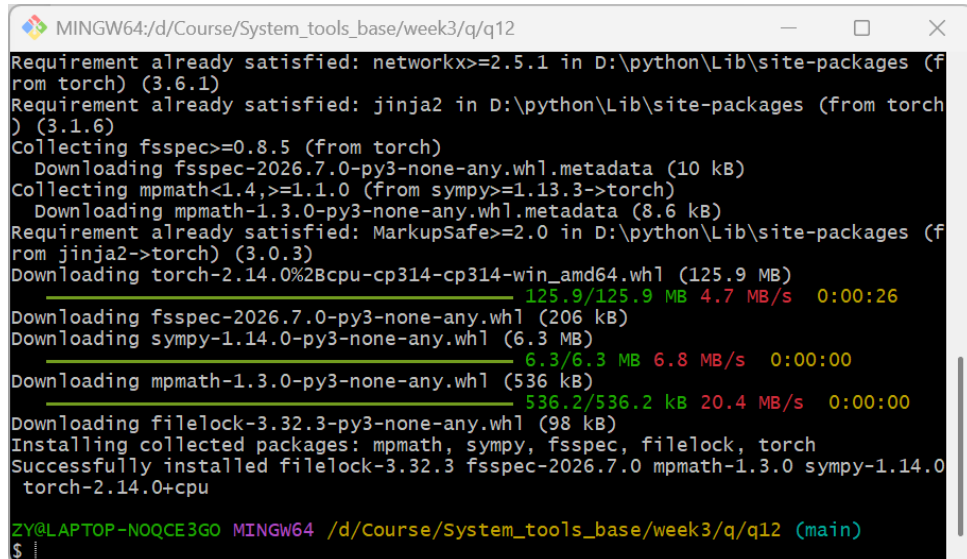
- 打印最终损失、weight 和 bias; 最终损失应小于 0.001。
- 不得直接把 weight 和 bias 赋值为 3 和 -1。

题目四题目要求

```
1 # TODO: 题目四代码汇总
2 # TODO: 清空梯度、反向传播、更新参数
3     opt.zero_grad()
4     loss.backward()
5     opt.step()
6
7 # TODO: 进入评估模式并在 no_grad 中打印最终损失、weight 和 bias
8 model.eval()
9 with torch.no_grad():
10     final_loss = loss_fn(model(x), y)
```

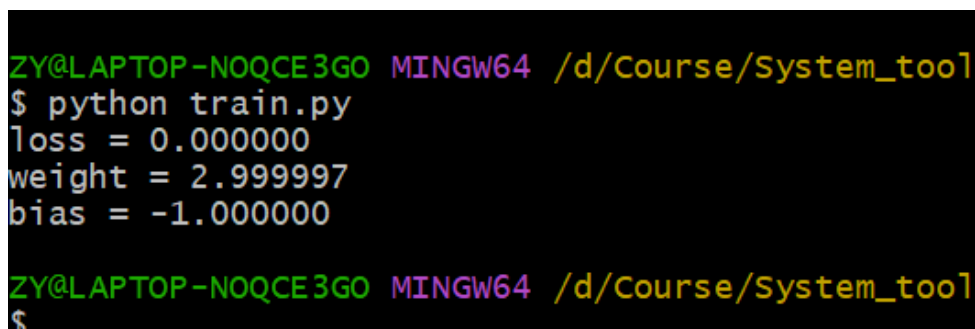
```
11 print(f"loss = {final_loss.item():.6f}")
12 print(f"weight = {model.weight.item():.6f}")
13 print(f"bias = {model.bias.item():.6f}")
```

部分实验过程截图



```
MINGW64:/d/Course/System_tools_base/week3/q/q12
Requirement already satisfied: networkx>=2.5.1 in D:\python\Lib\site-packages (from torch) (3.6.1)
Requirement already satisfied: jinja2 in D:\python\Lib\site-packages (from torch) (3.1.6)
Collecting fsspec>=0.8.5 (from torch)
  Downloading fsspec-2026.7.0-py3-none-any.whl.metadata (10 kB)
Collecting mpmath<1.4,>=1.1.0 (from sympy>=1.13.3->torch)
  Downloading mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
Requirement already satisfied: MarkupSafe>=2.0 in D:\python\Lib\site-packages (from jinja2->torch) (3.0.3)
Downloading torch-2.14.0%2Bcpu-cp314-cp314-win_amd64.whl (125.9 MB)
 125.9/125.9 MB 4.7 MB/s 0:00:26
Downloading fsspec-2026.7.0-py3-none-any.whl (206 kB)
Downloading sympy-1.14.0-py3-none-any.whl (6.3 MB)
 6.3/6.3 MB 6.8 MB/s 0:00:00
Downloading mpmath-1.3.0-py3-none-any.whl (536 kB)
 536.2/536.2 kB 20.4 MB/s 0:00:00
Downloading filelock-3.32.3-py3-none-any.whl (98 kB)
Installing collected packages: mpmath, sympy, fsspec, filelock, torch
Successfully installed filelock-3.32.3 fsspec-2026.7.0 mpmath-1.3.0 sympy-1.14.0 torch-2.14.0+cpu
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week3/q/q12 (main)
$
```

题目四实验过程截图：下载 torch



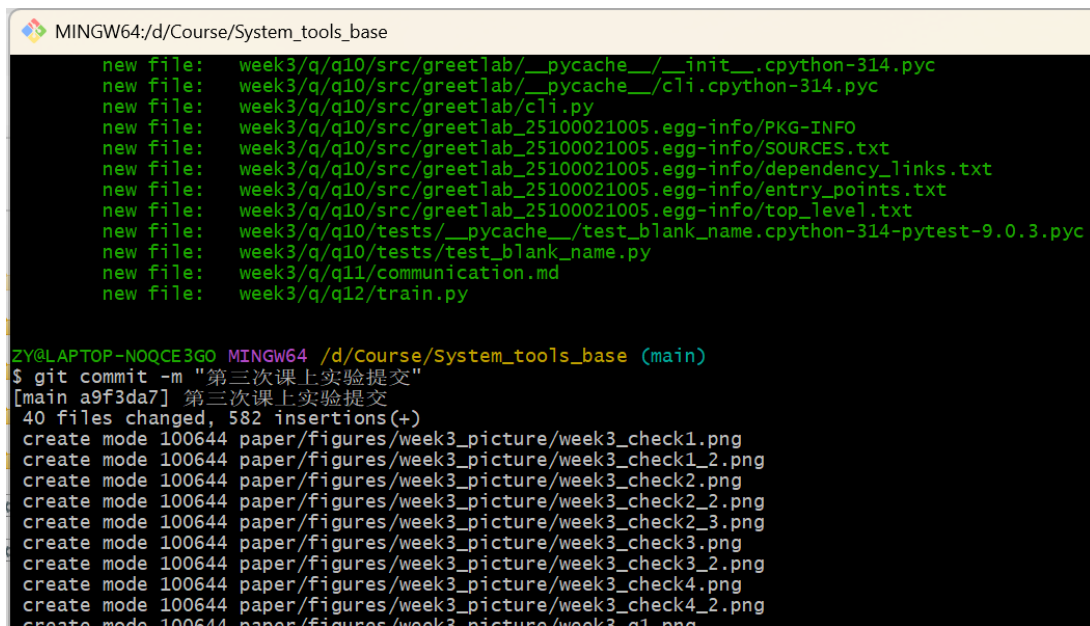
```
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week3/q/q12 (main)
$ python train.py
loss = 0.000000
weight = 2.999997
bias = -1.000000
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week3/q/q12 (main)
$
```

题目四实验过程截图：训练结果正确

实验结果

- 训练完成：200 轮 SGD 梯度下降，PyTorch CPU 版（torch 2.14.0+cpu）
- 收敛良好：loss = 0.000000（要求 < 0.001，达标）
- 参数正确：weight = 2.999997（3），bias = -1.000000（-1）
- 可复现：seed 固定，结果稳定

课上阶段提交记录



```

MINGW64:/d/Course/System_tools_base

new file:   week3/q/q10/src/greetlab/__pycache__/__init__.cpython-314.pyc
new file:   week3/q/q10/src/greetlab/__pycache__/cli.cpython-314.pyc
new file:   week3/q/q10/src/greetlab/cli.py
new file:   week3/q/q10/src/greetlab_25100021005.egg-info/PKG-INFO
new file:   week3/q/q10/src/greetlab_25100021005.egg-info/SOURCES.txt
new file:   week3/q/q10/src/greetlab_25100021005.egg-info/dependency_links.txt
new file:   week3/q/q10/src/greetlab_25100021005.egg-info/entry_points.txt
new file:   week3/q/q10/src/greetlab_25100021005.egg-info/top_level.txt
new file:   week3/q/q10/tests/__pycache__/test_blank_name.cpython-314-pytest-9.0.3.pyc
new file:   week3/q/q10/tests/test_blank_name.py
new file:   week3/q/q11/communication.md
new file:   week3/q/q12/train.py

ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base (main)
$ git commit -m "第三次课上实验提交"
[main a9f3da7] 第三次课上实验提交
40 files changed, 582 insertions(+)
create mode 100644 paper/figures/week3_picture/week3_check1.png
create mode 100644 paper/figures/week3_picture/week3_check1_2.png
create mode 100644 paper/figures/week3_picture/week3_check2.png
create mode 100644 paper/figures/week3_picture/week3_check2_2.png
create mode 100644 paper/figures/week3_picture/week3_check2_3.png
create mode 100644 paper/figures/week3_picture/week3_check3.png
create mode 100644 paper/figures/week3_picture/week3_check3_2.png
create mode 100644 paper/figures/week3_picture/week3_check4.png
create mode 100644 paper/figures/week3_picture/week3_check4_2.png
create mode 100644 paper/figures/week3_picture/week3_q1.png

```

课上阶段 commit 记录

2 课后练习与结果

实例 1

实验题目：创建一个带有的 Python 包，并在虚拟环境中安装。创建一个锁文件并检查它。

```

1 # TODO: 实例1代码汇总
2 # 环境检查
3 python --version
4 uv --version
5
6 # 1. 项目骨架
7 mkdir my_first_pkg && cd my_first_pkg
8 mkdir -p src/my_pkg tests
9 echo "# my-pkg" > README.md
10
11 # 2. 创建并激活虚拟环境
12 uv venv
13 source .venv/Scripts/activate
14
15 # 3. 安装依赖 + 生成锁文件 + 检查锁文件
16 uv sync

```

```

17 uv lock --check
18
19 # 4. 运行验证
20 uv run my-stats 1 2 3 4 5
21
22 # 5. 修复：安装 dev 组依赖后重跑测试
23 uv sync --extra dev
24 uv run pytest

```

运行结果：.venv 创建成功 (CPython 3.13.13)，实验输出：均值 3.0000 / 方差 2.0000 / 标准差 1.4142，5 个测试全部通过

```

(my_first_pkg)
ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week3/ex/ex01/my_first_pkg (main)
$ uv sync --extra dev
Resolved 13 packages in 1ms
Prepared 5 packages in 912ms
[0/6] Installing wheels...
warning: Failed to hardlink files; falling back to full copy. This may lead to degraded performance.
If the cache and target directories are on different filesystems, hardlinking may not be supported.
If this is intentional, set 'export UV_LINK_MODE=copy' or use '--link-mode=copy' to suppress this warning.
Installed 6 packages in 710ms
+ colorama==0.4.6
+ iniconfig==2.3.0
+ packaging==26.3
+ pluggy==1.6.0
+ pygments==2.11.0
+ pytest==9.1.1
(my_first_pkg)
ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week3/ex/ex01/my_first_pkg (main)
$ uv run my-stats 1 2 3 4 5
均值 : 3.0000
方差 : 2.0000
标准差: 1.4142
(my_first_pkg)
ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week3/ex/ex01/my_first_pkg (main)
$ uv run pytest
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.1.1, pluggy-1.6.0
rootdir: D:\course\System_tools_base\week3\ex\ex01\my_first_pkg
configfile: pyproject.toml
collected 5 items

tests\test_stats.py ..... [100%]

===== 5 passed in 0.07s =====
(my_first_pkg)
ZY@LAPTOP-NOQCE3G0 MINGW64 /d/Course/System_tools_base/week3/ex/ex01/my_first_pkg (main)
$ |

```

实例一运行结果

实例 2

实验题目：从零开始氛围编程一个小应用。不要手动编写一行代码。

```

1 # TODO: 实例2无代码汇总
2 #prompts 汇总:
3 D:\Course\System_tools_base\week3\ex\ex02, 根据题目要求,
4 在这个目录下工作。
5 题目: 从零开始氛围编程一个小应用。不要手动编写一行代码。
6 应用素材: 单位换算器, 用于长度 / 温度 / 重量多类型互转

```

运行结果：给 ai 简单的提示词，做出一个简易的单位换算器，运行结果正确，支持长度、温度、重量的互转。

```
MINGW64:/d/Course/System_tools_base/week3/ex/ex02
ZY@LAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week3/ex/ex02 (main)
$ python -X utf8 unit_converter.py < test_cases.txt
=====
      单位换算器 v1.0
=====
1.长度  2.温度  3.重量  q.退出
请选择换算类型 (1/2/3/q): 请输入数值: 输入单位 (米/千米/厘米/毫米/英里/英尺/英寸): 输出单位 (米/千米/厘米/毫米/英里/英尺/英寸):
>>> 1 千米 = 1000 米

1.长度  2.温度  3.重量  q.退出
请选择换算类型 (1/2/3/q): 请输入数值: 输入单位 (米/千米/厘米/毫米/英里/英尺/英寸): 输出单位 (米/千米/厘米/毫米/英里/英尺/英寸):
>>> 1 英里 = 1.60934 千米

1.长度  2.温度  3.重量  q.退出
请选择换算类型 (1/2/3/q): 请输入数值: 输入单位 (米/千米/厘米/毫米/英里/英尺/英寸): 输出单位 (米/千米/厘米/毫米/英里/英尺/英寸):
>>> 1 英尺 = 12 英寸

1.长度  2.温度  3.重量  q.退出
请选择换算类型 (1/2/3/q): 请输入数值: 输入单位 (摄氏度/华氏度/开尔文): 输出单位 (摄氏度/华氏度/开尔文):
>>> 0 摄氏度 = 32 华氏度
```

实例二运行结果

实例 3

实验题目：对比三个 1000+ star 的 GitHub 项目的 README。它们都同样有用吗？找出哪些内容在你看来主要是噪音，为你将来写 README 提供借鉴。

```
# TODO：实例3无代码汇总
```

根据推荐，选取三个风格差异大的知名项目做对比（超 1000 star）：

- fastapi（约 10.2 万 star）
- requests（约 5.4 万 star）
- linux（约 24 万 star）

实验结果：

三个项目的 README 虽然都远超 1000 star，但定位与有效信息密度差异显著，并非“同样有用”。对比见表 1。

表 1: 三个项目 README 对比（star 数为 2026 年 9 月数据）

项目	star 数	README 规模	README 类型
fastapi	约 10.2 万	约 520 行	营销转化型
requests	约 5.4 万	约 78 行	实用克制型
linux	约 24 万	约 18 行	极简导航型

结论：fastapi 的 README 有效信息占比仅约三成，大量篇幅属于噪音；requests 的 README 短小精悍、几乎全部有效，是普通项目的最佳范本；linux 的 README 虽无噪音，但信息量近乎为零，仅充当文档导航入口，其极端风格不适合普通项目模仿。

主要噪音内容（集中体现在 fastapi 的 README 中）：

1. 赞助商广告：16 个以上赞助商 logo，约占全文三分之一篇幅，对使用者无价值；
2. 名人背书：引用 Microsoft、Uber、Netflix、Cisco 等公司员工的夸赞语录，无法验证；
3. 不可验证的夸张数字：如“开发提速 200%-300%”、“bug 减少 40%”，脚注承认仅为内部团队估计；
4. 兄弟产品与纪录片推广：交叉销售 Typer、FastAPI Cloud 等内容，与当前项目使用无关。

为将来编写 README 提供的借鉴：

1. 第一屏给出“一句话定位 + 最小可运行示例”，让读者 30 秒内建立信任；
2. 社会证明使用可验证数据（如下载量、依赖仓库数），而非名人名言；
3. 安装命令放在靠前位置，一句话即可；
4. 详细教程放文档站，README 只做入口，避免篇幅过长；
5. 徽章数量控制在 3-5 个以内，超过即为噪音；
6. 真实踩坑提示（如克隆命令的注意事项）很有价值；

实例 4

实验题目：浏览一个知名项目（例如 Redis 或 curl）的源码。找出讲义中提到的几种注释类型：有用的 TODO、指向外部文档的引用、解释为何避开某种做法的“why not”注释、或血泪教训。如果这些注释不存在，会失去什么？

```
1 # TODO: 实例4无代码汇总
```

结果：

本次浏览了 Redis (<https://github.com/redis/redis>) unstable 分支的 14 个核心源文件及 redis.conf，四类注释均有发现；其中字面 TODO：注释为 0 处，Redis 团队改用“已知局限 + 优化方向”的完整句注释承担同样职责（如 ae.c 明确列出定时器 O(N) 遍历的两种优化方案，并注明“Redis 目前不需要”）。四类注释及其缺失后果见下表。

注释类型	Redis 实例（文件）	若缺失会失去什么
有用的 TODO / 局限说明	ae.c: 定时器 $O(N)$ 遍历 + 优化方向	已知局限被当缺陷“优化”而引入 bug
指向外部文档的引用	hyperloglog.c: 论文 [1][2]、arXiv:1702.01284; redis.conf: 官方文档	算法正确性与误差边界无法追溯验证
why not 注释	server.c: 存盘期间禁 rehash (COW); dict.c: 空桶扫描上限	有意的设计被“修复”，引发 COW 页风暴或主线程长阻塞
血泪教训	redis.conf: fsync 阻塞、30 秒丢失窗口、AOF 截断与 ext4	已知故障被重复踩坑，甚至被当成新 bug 错误“修复”

结论：这些注释承载的是“决策上下文”——代码为什么长这样、已知代价是什么、哪些方案被否决过。缺少它们，代码仍能运行，但维护者会失去判断依据：把有意的设计当成缺陷，把已知的坑当成新问题，修复与优化的成本都会显著上升。

实例 5

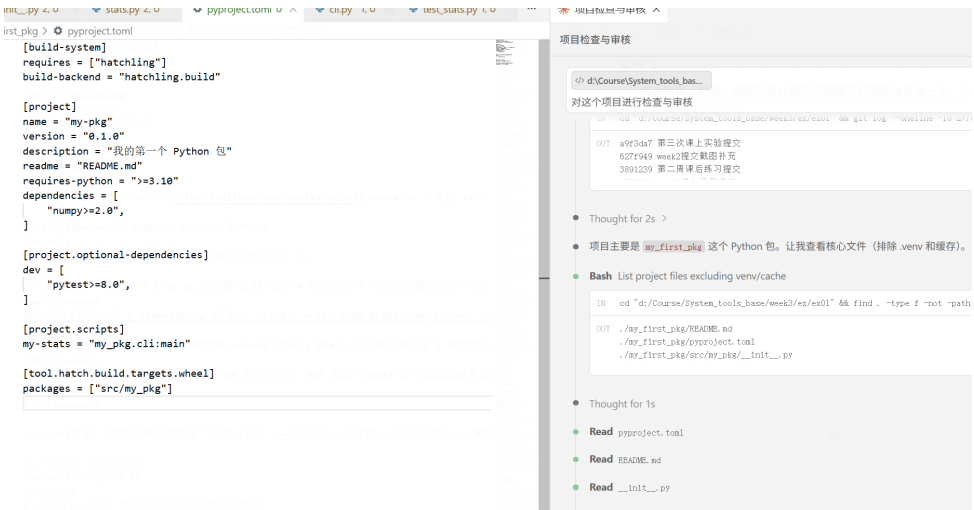
实验题目：把同一编程任务分别用内联聊天和智能体各做一次，比较体验差异。

1 # TODO: 实例 5 无代码汇总

此题复用实例 1 案例，分别用内联聊天和智能体完成同一任务（创建 Python 包、构建 wheel、安装并验证），比较体验差异。

体验差异：

1. 内联聊天每步都要人复制、运行、发现问题再追问，步骤多、节奏慢，但每一步演化都看得见，学习价值高，适合初学者理解代码怎么一步步变完整；
2. 智能体把“写一跑一验一改”的循环全部接管，交付快、结果完整，但过程近乎黑盒，适合熟练者快速产出完整任务；
3. 出错机制不同：内联聊天的错误要人发现后带着报错去追问；智能体会自己读报错并修正，人工介入点明显更少；



实例五内联聊天



实例五智能体编程

实例 6

实验题目：用 AI 编程智能体浏览一个陌生的代码库，并让他解释其中部分功能

TODO：实例6无代码汇总

ai 输出功能解释：

本实例选择 curl 的 URL 解析 API (lib/urlapi.c) 作为解释对象。该模块把 URL 字符串拆解后存入 CURLU 句柄，句柄内含 scheme、user、password、host、port、path、

query、fragment 等部件；对外通过 `curl_url_get()` 与 `curl_url_set()` 按 `CURLUPART_*` 枚举读写单个部件，并用 `flags` 控制行为（如隐藏默认端口、追加查询串）。解析过程分为四阶段：先识别 `scheme`（缺省时默认 `https`，或按主机名前缀如 `ftp` 猜测）；再解析 `authority`，依次抽出 `user:password@`、端口与主机名，端口支持 IPv6 的 `[]` 括法，冒号后无数字时按浏览器行为忽略；随后对主机名归一化，IPv4 简写如 `16843009`、`0x7f` 统一转为点分十进制，IPv6 规范化并提取 `zone id`；最后切分路径、查询与片段，并按 RFC 3986 5.2.4 清理路径中的 `./` 与 `../` 段。全程以 `CURLUE_*` 错误码返回失败原因，并拒绝含控制字符的输入、限制 `file:` URL 只能指向本机，以防御注入与 UNC 路径攻击。

[明确 AI 编程智能体任务 >](#)

进入实例 6。题目要点是“用 AI 智能体浏览陌生代码库 + 解释部分功能”。我选 `curl`（题目点名的项目，与上题 `redis` 形成互补，且是 C 语言代码库，对我们是“陌生”的）。先以智能体方式真实浏览——获取仓库结构并定位核心模块：

[浏览代码库内容 >](#)

实例六运行结果

实例 7

实验题目：大多数编程智能体都支持某种“YOLO 模式”（例如，在 Claude Code 中是 `--dangerously-skip-permissions`）。直接使用此模式并不安全，但在虚拟机或容器等隔离环境中运行编程智能体然后启用自主操作可能是可以接受的。在你的机器上搭建这样的环境。

```

1 # TODO: 实例7代码汇总
2 # 1. 启动 Docker Desktop 并确认引擎
3 docker version
4
5 # 2. 拉取轻量 Python 镜像
6 docker pull python:3.12-slim
7
8 # 3. 创建后台容器 (sleep infinity 保持运行)
9 docker run -d --name yolo-sandbox python:3.12-slim sleep infinity
10
11 # 4. 容器内安装编程智能体 Aider
12 docker exec yolo-sandbox pip install --no-cache-dir aider-chat
13
14 # 5. 确认 YOLO 模式开关存在
15 docker exec yolo-sandbox aider --help | Select-String "yes-always"
16 # --yes-always Always say yes to every confirmation

```

```

17
18 # 6. 容器内模拟 YOLO 模式危险操作（无确认删除文件）
19 docker exec yolo-sandbox sh -c "mkdir -p /work && echo secret-token
    -123 > /work/secret.txt && rm -f /work/secret.txt"
20
21 # 7. 验证宿主不受影响
22 Test-Path C:\work\secret.txt          # 输出 False
23
24 # 8. 出问题直接丢弃重建（新容器 ID 与旧的不同）
25 docker rm -f yolo-sandbox
26 docker run -d --name yolo-sandbox python:3.12-slim sleep infinity

```

运行结果：本机已安装 Docker Desktop (29.5.2, WSL2 后端)，故采用容器方案：拉取 python:3.12-slim 镜像，创建 yolo-sandbox 容器，在容器内安装支持 --yes-always（等价于 YOLO 模式）的编程智能体 Aider，并验证隔离性。

```

Step 1/3  env: container + agent + YOLO switch

PS C:\> docker run -d --name yolo-sandbox python:3.12-slim sleep infinity
64ffe320a3052754bc75a8c8949efeb01b3fe2d517df4120ef6d27603075785c

PS C:\> docker ps --filter name=yolo-sandbox --format "{{.Names}} / {{.Image}}"
yolo-sandbox / python:3.12-slim

PS C:\> docker exec yolo-sandbox pip install --no-cache-dir -q aider-chat
installed successfully (exit code 0)

PS C:\> docker exec yolo-sandbox aider --version
aider 0.86.2

PS C:\> docker exec yolo-sandbox aider --help | Select-String "yes-always"
--yes-always          Always say yes to every confirmation [env var:
-> YOLO mode: skip all confirmations, act directly

```

实例七环境搭建

实例 8

实验题目：试着写一个正则表达式，并使用 grep 命令行工具在你的代码中查找 subprocess.Popen(..., shell=True) 的出现位置。

```

1 # TODO: 实例8代码汇总
2 # 1. 朴素搜索（观察误报，预期命中 8 行）
3 grep -n "shell=True" sample_shell.py
4
5 # 2. ERE 单行版：Popen 参数到第一个 ) 前的 shell=True
6 grep -nE 'subprocess\.Popen\[([^\)]*\)shell[[:space:]]*=[[:space:]]*True'
    sample_shell.py
7
8 # 3. PCRE 单行版：非贪婪 .*?（需先设置 UTF-8 locale）

```

```

9 export LC_ALL=C.UTF-8
10 grep -nP 'subprocess\.Popen\(.*?shell\s*=\s*True' sample_shell.py
11
12 # 4. PCRE 多行版: (?s:.*) 让 . 匹配换行, 可跨行匹配
13 grep -nP 'subprocess\.Popen\((?s:.*)shell\s*=\s*True' sample_shell.
    py
14
15 # 5. 递归审计真实代码目录 (排除 .venv 与演示目录 ex08)
16 grep -rnP 'subprocess\.Popen\((?s:.*)shell\s*=\s*True' \
17 /d/Course/System_tools_base/week3/ex \
18 --include='*.py' --exclude-dir=.venv --exclude-dir=ex08
19 echo $? # 预期 1 = 无匹配

```

运行结果:

```

MINGW64/d/Course/System_tools_base/week3/ex/ex08
$ grep -nP 'subprocess\.Popen\(.*?shell\s*=\s*True' sample_shell.py
9:subprocess.Popen("ls -l", shell=True) # 3. 单行 shell=True
10:subprocess.Popen("dir", shell = True) # 4. 参数带空格 shell = True
11:subprocess.Popen( # 5. 多行调用, shell=True 在下一行
15:subprocess.Popen("rm -rf /tmp/x", shell=True, cwd="/tmp") # 6. shell=True 后面还有参数
21:doc = "subprocess.Popen(cmd, shell=True) # 字符串里也有"

ZYBLAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week3/ex/ex08 (main)
$ grep -nP 'subprocess\.Popen\((?s:.*)shell\s*=\s*True' sample_shell.py
9:subprocess.Popen("ls -l", shell=True) # 3. 单行 shell=True
10:subprocess.Popen("dir", shell = True) # 4. 参数带空格 shell = True
11:subprocess.Popen( # 5. 多行调用, shell=True 在下一行
15:subprocess.Popen("rm -rf /tmp/x", shell=True, cwd="/tmp") # 6. shell=True 后面还有参数
21:doc = "subprocess.Popen(cmd, shell=True) # 字符串里也有"

ZYBLAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week3/ex/ex08 (main)
$ grep -rnP 'subprocess\.Popen\((?s:.*)shell\s*=\s*True' /d/Course/System_tools_base/week3/ex --include='*.py' --exclud
e-dir=.venv
/d/Course/System_tools_base/week3/ex/ex08/sample_shell.py:9:subprocess.Popen("ls -l", shell=True) # 3. 单
行 shell=True
/d/Course/System_tools_base/week3/ex/ex08/sample_shell.py:10:subprocess.Popen("dir", shell = True) # 4.
参数带空格 shell = True
/d/Course/System_tools_base/week3/ex/ex08/sample_shell.py:11:subprocess.Popen( # 5.
多行调用, shell=True 在下一行
/d/Course/System_tools_base/week3/ex/ex08/sample_shell.py:15:subprocess.Popen("rm -rf /tmp/x", shell=True, cwd="/tmp")
# 6. shell=True 后面还有参数
/d/Course/System_tools_base/week3/ex/ex08/sample_shell.py:21:doc = "subprocess.Popen(cmd, shell=True) # 字符串里也有"

ZYBLAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week3/ex/ex08 (main)
$ echo $?
1

ZYBLAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week3/ex/ex08 (main)
$ grep -rnP 'subprocess\.Popen\((?s:.*)shell\s*=\s*True' /d/Course/System_tools_base/week3/ex --include='*.py' --exclud
e-dir=.venv --exclude-dir=ex08

ZYBLAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week3/ex/ex08 (main)
$ echo $?
1

ZYBLAPTOP-NOQCE3GO MINGW64 /d/Course/System_tools_base/week3/ex/ex08 (main)
$ |

```

实例八运行结果

实例 9

实验题目: 在你的 IDE 或文本编辑器里练习 regex 查找替换, 把这些讲义中的 - Markdown 项目符号标记替换为 * 项目符号标记。

1 # TODO: 实例9无代码汇总

运行结果: 正则替换如下图所示, 所有 Markdown 项目符号 - 均被替换为 *, 且未误伤其他内容。



实例九运行结果

实例 10

实验题目：安装 Docker，用它用 docker compose 本地搭建 missing-semester 课程网站。

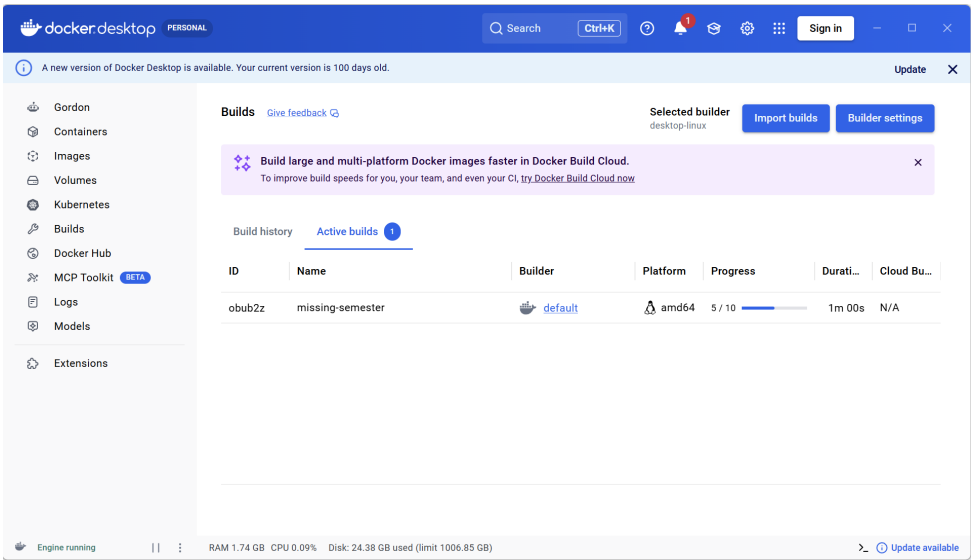
```

1 # TODO: 实例10代码汇总
2 docker ps # 确认引擎
3 git clone --depth 1 https://github.com/missing-semester/missing-semester.git
4 docker compose up --build -d # 构建并后台启动
5 docker compose ps # 看容器状态
6 curl -s -o /dev/null -w "%{http_code}" http://localhost:4000/ # 200
7 docker compose down # 停止

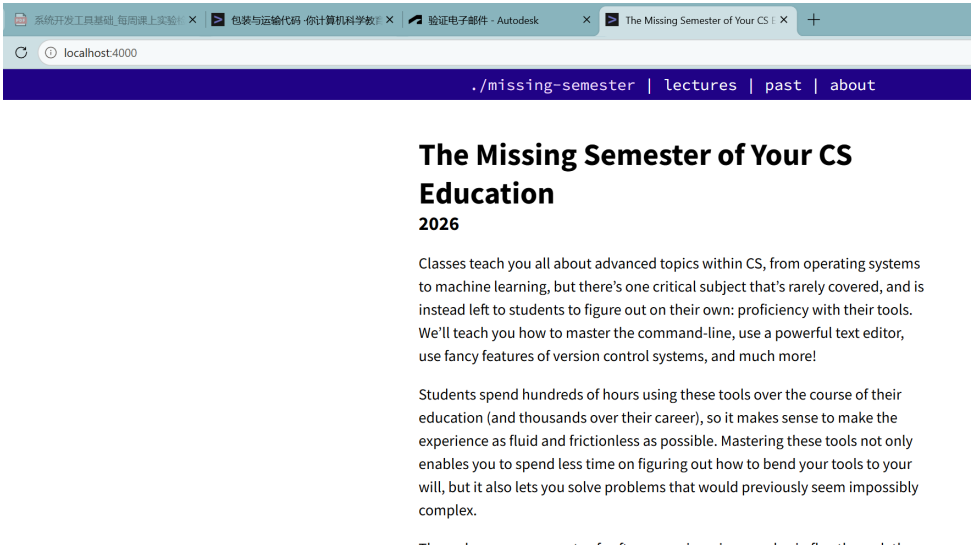
```

运行结果：

- 容器 missing-semester-server-1 运行正常，0.0.0.0:4000->4000/tcp 映射成功
- Jekyll 站点构建 4.2 秒完成，日志显示 Server running... http://0.0.0.0:4000
- 浏览器打开 http://localhost:4000，首页标题确认是“The Missing Semester of Your CS Education”

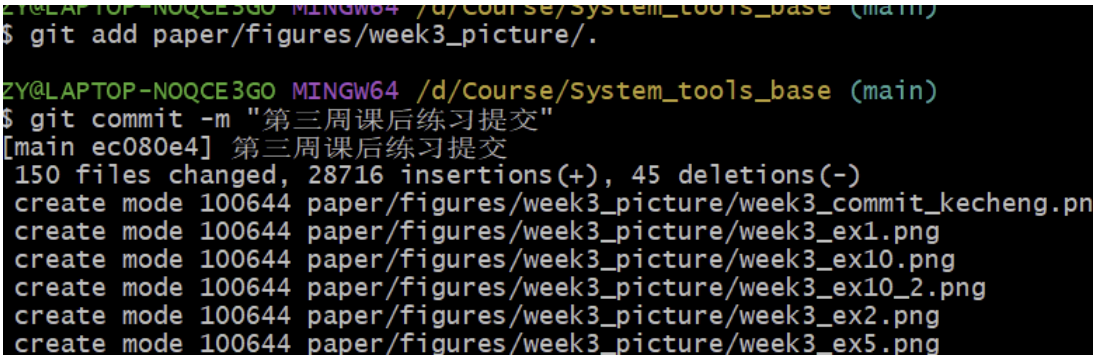


实例十运行结果：搭建中



实例十运行结果：本地运行成功

课后阶段提交记录



课后阶段 commit 记录

3 解题感悟

做完这一系列练习，最大的体会是：工具本身并不难，难的是理解每个工具“为什么存在”。命令行、正则、包管理、容器这些平日里被图形界面藏起来的東西，恰恰是工程世界里最通用、最持久的能力——它们不依赖某个软件的版本，学会了就一直在。

过程中的几次“翻车”也让我明白，调试和排错才是学习的真正主体：报错不可怕，可怕的是不读报错、不查文档、不验证结果。另一个深刻的感受是，AI 编程助手正在大幅降低入门门槛，但它不会替你建立判断力：一个命令该不该执行、一个正则会不会误伤、一个环境是否隔离，这些安全意识依然要自己长出来。工具是手段，解决问题的思路才是目的。